Making a codebase agent-ready

Skills, rules, hooks, subagents, and external tools

Skills **Rules** **Hooks**

Subagents Model Context Protocol Cloud agents

What Are Skills?

A skill teaches an agent how to perform a specific task.

01

Instructions

A repeatable procedure

02

Resources

Scripts, references, and templates

03

Storage

Files tracked with the project

Skills use an open standard supported by multiple agents.

Skill Types

01	**Built-in or custom**	Provided by Cursor, or written for your work
02	**Project or user**	Stored in a repository, or in your user directory
03	**Automatic or explicit**	Selected by relevance, or invoked with /skill-name
04	**Nested project skills**	Available within a particular project directory

These describe origin, location, and invocation separately.

How Skills Work

01

Discover

Cursor finds available skills.

02

Select

The agent uses the name
and description.

03

Read

The selected skill's instructions enter context.

04

Use resources

Supporting files load when needed.

/skill-name attaches to one message. A Custom Mode lasts for the session.

Skill Anatomy

`.cursor/skills/add-endpoint/SKILL.md`

```
---
name: add-endpoint
description: Add an endpoint and tests.
---

1. Read the task and product intent.
2. Write a failing behavioral test.
3. Make the smallest change.
4. Run validation.
```

Frontmatter

name and description identify the skill.

Procedure

The body gives the working steps.

Supporting files

Optional scripts/, references/, and assets/.

Optional fields include paths and disable-model-invocation.

Skill Examples

01	**add-endpoint**	Read requirements, add tests, implement, validate
02	**fix-bug**	Reproduce the defect and add a regression test
03	**deploy-app**	Follow a deployment procedure using a script
04	**File-specific skill**	Use paths to limit relevant file patterns

The lab skills prescribe a test-first procedure. They do not enforce compliance.

What Are Rules?

Rules supply reusable instructions in the agent’s context.

01

Project knowledge

Conventions and architectural constraints

02

Relevant context

Instructions that apply to the current work

03

Shared guidance

Files and settings used across sessions

A rule guides the model. It does not execute a check.

Rule Types

01	**Project Rules**	.cursor/rules/*.mdc
02	**User Rules**	Personal preferences across projects
03	**Team Rules**	Shared instructions managed in the dashboard
04	**AGENTS.md**	Plain Markdown project instructions

Nested AGENTS.md files combine with parent instructions.

How Rules Work

01

Always

Included in every chat session.

02

Intelligent

The agent selects by the description.

03

File scoped

A matching file is in context.

04

Manual

Attach with @rule-name.

When a rule applies, its contents enter the model’s context.

Rule Anatomy

`.cursor/rules/api.mdc`

```
---
description: Ticket route conventions
globs: app/routes/**/*.py
alwaysApply: false
---

- Validate request data with models.
- Keep error responses consistent.
- Test changed behavior.
```

description

Explains the rule's purpose.

globs

Matches files and directories.

alwaysApply

Controls unconditional loading.

Project rules require .mdc. A plain .md file in this folder is ignored.

Rule Examples

01	**Application rule**	Route and model constraints for matching files
02	**Test rule**	Public behavior tests and storage reset
03	**Always-applied rule**	A small set of project-wide instructions
04	**Manual rule**	Explicit guidance for an occasional migration

Keep rules focused. Use formatters and linters for mechanical style checks.

Rules and Skills

Rules

State constraints and conventions.

Example
Keep ticket error bodies consistent.

Load always, by files, by relevance,
or through an explicit mention.

Skills

Describe a task procedure.

Example
Follow the endpoint workflow.

Load when selected or invoked.
Can package supporting resources.

Both guide the model. Neither guarantees that the result satisfies the instructions.

What Are Hooks?

Hooks run at defined events in the agent’s work.

01

Observe

Record what happened

02

Act

Run a script after an event

03

Control

Allow or block supported actions

The host invokes a registered hook when its event occurs.

Hook Types

Execution types

Command-based
Runs a program.

Prompt-based
Uses a model to assess a condition.

Event categories

Agent
During an agent session.

Tab and application lifecycle
During completion or workspace events.

Project and user configuration have different scope. Cloud support varies.

How Hooks Work

01

Event

A configured event occurs.

02

Input

The host sends event data.

03

Execution

The command reads and evaluates it.

04

Response

The host handles the result.

Command hooks exchange JSON through standard input and output.

Hook Anatomy

`.cursor/hooks.json`

```
{
  "version": 1,
  "hooks": {
    "stop": [{
      "command": ".cursor/hooks/validate.sh",
      "timeout": 60,
      "loop_limit": 2
    }]
  }
}
```

Event

stop selects the completion event.

Command

The host starts this script.

Options

timeout and loop_limit bound execution.

The event determines the script’s input and allowed response fields.

Hook Examples

01	**After a file edit**	Run formatting
02	**Before a shell command**	Check whether the command is allowed
03	**After a completed turn**	Run the lab validation script
04	**On validation failure**	Request another attempt with followup_message

The lab caps follow-ups at two. Failing validation can remain after that limit.

Rules and Hooks

Rule

Instruction the model reads.

Example
Do not run git push.

Compliance depends on the agent following the instruction.

Hook

Event-triggered execution.

Example
A shell permission hook rejects a prohibited command.

Coverage and failure settings matter.

Permissions and other access controls remain necessary for security boundaries.

What Are Subagents?

A subagent handles delegated work in a separate context.

01

Assignment

A bounded task from the parent agent

02

Separate work

Its own instructions and context window

03

Result

A final report returned to the parent

The parent supplies the relevant information for the task.

Subagent Types

Built-in and custom

Explore
Searches and analyzes code.

Browser
Handles browser interactions.

Custom agents can be project or user scoped.

Foreground and background

Foreground
The parent waits for the result.

Background
Work continues independently.

The type of task and execution mode are separate choices.

How Subagents Work

01

Assign

The parent provides the task and context.

02

Work

The subagent uses a separate context.

03

Report

The result returns to the parent.

04

Continue

The parent uses the findings.

Delegation may happen automatically or through an explicit request.

Subagent Anatomy

`.cursor/agents/reviewer.md`

```
---
name: reviewer
description: Review a completed change.
model: inherit
readonly: true
is_background: false
---

Read the task and diff.
Check tests and validation evidence.
Return ACCEPT or REVISE with reasons.
```

Identity

name and description support selection.

Execution

model, readonly, and is_background.

Instructions

Define the task and expected report.

Project agents live in .cursor/agents/. User agents live under ~/.cursor/agents/.

Subagent Examples

01	**Planner**	Identify affected files, tests, and risks
02	**Reviewer**	Check the diff against requirements and evidence
03	**Verifier**	Run checks and report what passed or remains
04	**Parallel investigation**	Assign independent questions to separate contexts

A focused assignment and a defined report make the returned result useful.

What Is the Model Context Protocol?

MCP connects agents to external tools and data through a shared protocol.

01

Client

The connection used by the agent’s host

02

Server

Exposes supported capabilities

03

External system

Provides the underlying data or operation

Available capabilities depend on the server and the access you grant.

MCP Connection Types

01	**Standard input/output**	A local process started with a command
02	**Server-sent events**	A connection to a server endpoint
03	**Streamable HTTP**	An HTTP connection to a server endpoint

Servers can expose tools, prompts, resources, and additional supported capabilities.

How MCP Works

01

Connect

Configure a server and authenticate.

02

Discover

The host receives available capabilities.

03

Call

The agent selects a relevant tool.

04

Return

The tool supplies a result.

Cursor asks for tool approval by default. Run Mode settings affect approval.

MCP Configuration Anatomy

`.cursor/mcp.json`

```
{
  "mcpServers": {
    "local-example": {
      "command": "python",
      "args": ["server.py"]
    },
    "remote-example": {
      "url": "https://example.com/mcp"
    }
  }
}
```

Server name

A label inside mcpServers.

Local process

command, args, and optional env.

Remote connection

url, with headers or auth as needed.

Project configuration uses .cursor/mcp.json. Global configuration uses ~/.cursor/mcp.json.

MCP Examples

01	**Issue tracking**	Find a Linear issue and read its requirements
02	**Design context**	Retrieve information from Figma
03	**Browser tools**	Inspect an interface or test a user flow
04	**Development tools**	Use a server for a supported development system

Choose trusted servers and give each connection only the access it needs.

APIs, MCP, and Skills

01

APIs

Application programming interface

A service exposes operations. Your code calls them.

02

MCP

Model Context Protocol

A server exposes capabilities through a common agent protocol.

03

Skills

Task procedures

Instructions explain how to complete work using available tools.

A skill can use an MCP tool. That server may call an application's API.

MCP Servers and Uses

Issue tracking	Jira / Linear	Read requirements and manage issues.
Documentation	Notion / Confluence	Find specifications and project knowledge.
Application monitoring	Datadog / Sentry	Inspect errors and runtime information.
Databases	Amazon Redshift	Explore schemas and query permitted data.
Technical references	Context7	Retrieve library documentation and examples.
Experiment tracking	Arize	Inspect traces and evaluation results.

Available operations depend on the server, account, and permissions.

Cloud Agent Environment Setup

01

Configure

Set up the cloud execution environment.

02

Install

Provide dependencies and required runtimes.

03

Run

Confirm the project's commands work.

Environment setup comes before assigning implementation work.

Cloud Agent Access

01

Secrets

Supply required credentials through the supported secret store.

02

Network

Permit the endpoints the workflow needs.

03

Local testing

Avoid dependencies on unreachable services.

For cloud roles, prefer short-lived identity tokens when supported.

Skills and AGENTS.md for Cloud Agents

AGENTS.md

Project orientation

Commands to run and test services.
Links to testing and debugging notes.

Skills

Detailed procedures

Service-specific debugging.
Setup for dependencies.
Repeatable validation steps.

Commit project instructions and skills with the repository.

Rules for Cloud Agents

01

User

Personal instructions across repositories.

02

Team

Shared conventions across team members.

03

Repository

Project-specific rules committed in .cursor/rules/.

Place instructions at the level where they apply.

Tools for Cloud Agents

01	**Required systems**	Identify what a developer needs to complete the task
02	**Connections**	Provide supported tools or MCP servers
03	**Permissions**	Allow the operations the task requires
04	**Verification**	Confirm the agent can use each required tool

Missing tool access can prevent the agent from completing otherwise clear work.

Tools the Agent Can Use Reliably

01

Observe

Look at how the agent uses the tool.

02

Identify problems

Find missed arguments or distracting output.

03

Improve

Adjust the interface and try again.

Clear arguments and useful output make tools easier to use.

Project Assignment

01

Describe

Write AGENTS.md and product boundaries.

02

Guide

Add one rule and one skill.

03

Validate

Create a shared validation script.

04

Demonstrate

Show activation and a check failure.

Explain what loads, what runs, and what happens when validation fails.